

Boolean Information Retrieval System

Information Retrieval – Project Report

Luca Iacopino – 564278

Academic Year 2024/2025

1 Introduction

The goal of this project is to build a simple Information Retrieval system. The system creates an inverted index over a collection of documents, stores it on disk, and then answers Boolean queries written by the user from the command line.

The collection used is **Reuters-21578**, a classic dataset of 21,578 news articles published by Reuters in 1987. It is small enough to be indexed in a few seconds, but large enough to have long postings lists, which is useful for the experiments.

The system is written in Python. Everything related to Information Retrieval (tokenization, index, queries, skip pointers, edit distance) is implemented from scratch, without external libraries.

Besides the two mandatory functions (indexing and retrieving), the following features have been implemented:

- **Group A1:** conjunctive and disjunctive queries with more terms;
- **Group A2:** optimized conjunctive queries (terms processed by increasing document frequency);
- **Group A4:** stop list;
- **Group A7:** edit distance to suggest alternatives for misspelled terms;
- **Group B2:** skip pointers in postings lists, with experiments on execution time.

2 System Architecture

The system works in two separate phases:

1. **Indexing** (run once, in advance): the documents are read, tokenized and used to build the inverted index, which is saved on disk.
2. **Retrieving:** the index is loaded and the user can type queries.

The code is split into small modules, each one with a single task (Table 1).

The parser is the only part that depends on the dataset. It turns the Reuters files (SGML format) into simple (docID, text) pairs, and all the other modules only work on these pairs. To use a different collection, it is enough to write a new parser.

Document unit. Each news article is one document. The docID is the **NEWID** attribute of the article, which is unique and goes from 1 to 21,578. The text of a document is the content of its **TEXT** tag (title, dateline and body). The other fields (topics, places, etc.) are metadata and are ignored.

Module	Task
<code>parser.py</code>	reads the dataset files and returns (docID, text) pairs
<code>tokenizer.py</code>	splits a text into normalized terms
<code>stoplist.py</code>	list of stop words
<code>indexer.py</code>	builds the inverted index in memory
<code>storage.py</code>	saves the index on disk and reads it back
<code>boolean.py</code>	intersection and union of postings lists
<code>skiplist.py</code>	postings lists with skip pointers
<code>query.py</code>	parses the user query
<code>searcher.py</code>	runs the queries on the index
<code>spelling.py</code>	edit distance and spelling suggestions

Table 1: Modules of the system.

3 Text Processing

The same function is used for documents and queries, so that a query term always matches the form stored in the index. The steps are:

1. all letters are turned into **lowercase**;
2. **accents** are removed (e.g. *café* → *cafe*);
3. a **token** is a sequence of letters and/or digits; everything else (spaces, punctuation, symbols) is a separator.

Stop list. The 25 stop words of the course textbook (Manning et al., *Introduction to Information Retrieval*, Figure 2.5) are used: *a, an, and, are, as, at, be, by, for, from, has, he, in, is, it, its, of, on, that, the, to, was, were, will, with*. These terms are dropped at indexing time and ignored at query time.

4 Data Structures

4.1 Dictionary

In memory, during indexing, the dictionary is a Python `dict`, which is a **hash table**. Each term is a key and its value is the postings list of the term. Looking up a term takes $O(1)$ time on average. A hash table does not allow searching for all terms with a given prefix, but this is not needed since the system does not support wildcard queries.

On disk, the dictionary is stored in the text file `dictionary.txt`. Terms are sorted alphabetically and each line has three fields:

```
term    document_frequency    offset
```

The **offset** is the position (in bytes) where the postings list of the term starts in the postings file. It works as the “pointer to the postings list” seen in class.

At search time only the dictionary is loaded in memory (again as a hash table: $\text{term} \rightarrow (\text{df}, \text{offset})$). The final dictionary has 48,404 terms and takes 849 KB.

4.2 Postings Lists

In memory, a postings list is a Python `list` (a dynamic array) of docIDs, sorted in increasing order. Each document appears only once in the list of a term. The document frequency of a term is simply the length of its list, so it does not need to be stored separately in memory.

On disk, all postings lists are written one after the other in the binary file `postings.bin`. Each docID is stored as a 4-byte unsigned integer. To read the list of a term, the system:

1. looks up the term in the dictionary and gets its *df* and offset;
2. moves to the offset in the postings file;
3. reads $df \times 4$ bytes and decodes them into a list of integers.

In this way a postings list is read from disk only when a query needs it. The postings file takes 5.9 MB.

<code>dictionary.txt</code>	<code>postings.bin</code>
<code>...</code>	
<code>cocoa 129 <offset> -----></code>	<code>[1, 275, 1889, 2521, ...] (129 x 4 bytes)</code>
<code>...</code>	

Figure 1: Link between the dictionary and the postings file.

4.3 Skip Lists

A postings list with skip pointers is represented by the class `SkipList`, which has two lists of the same length:

- **postings**: the sorted docIDs;
- **skips**: `skips[i]` is the position reached by the skip pointer stored at position `i`, or `None` if position `i` has no skip pointer.

Following the heuristic seen in class, a list of length P has about $\sqrt{P}$ evenly spaced skip pointers, so the span between two pointers is $s = \lfloor \sqrt{P} \rfloor$. Pointers are placed at positions $0, s, 2s, \dots$ (only if the target position exists). The span can also be fixed by hand, which is used in the experiments.

For example, the list `[2, 4, 8, 16, 19, 23, 28, 43]` has 8 elements, so $s = 2$ and:

`skips = [2, None, 4, None, 6, None, None, None]`

Since the pointers are evenly spaced, they could also be computed on the fly ($i + s$). We chose to store them explicitly to keep the structure clear; the cost is one extra entry per element. Skip pointers are not saved on disk: they are built when a list is used in a conjunctive query. This keeps the file format simple.

5 Algorithms

5.1 Index Construction

The index is built in a single pass over the collection:

1. for each document, the text is tokenized and turned into a **set** of terms (so each term is counted once per document), minus the stop words;
2. for each term, the docID is appended to its postings list in the hash table (if the term is new, an empty list is created first);
3. at the end, every postings list is sorted;

4. the dictionary (sorted alphabetically) and the postings lists are written on disk.

Indexing the whole collection takes about 3.4 seconds. The whole index fits in memory, which is fine for a collection of this size; for a much larger collection a block-based algorithm such as BSBI or SPIMI would be needed.

5.2 Query Parsing

The supported queries are a single term, a conjunctive query (t_1 AND t_2 AND ... AND t_n) or a disjunctive query (t_1 OR t_2 OR ... OR t_n). Operators must be written in uppercase; this is checked before tokenization, otherwise AND would be turned into the word *and*. AND and OR cannot be mixed in the same query. Each term goes through the same tokenizer used for the documents, and stop words are removed. Wrong queries (e.g. a missing term, or mixed operators) produce an error message.

5.3 Intersection and Union

Both operations use the two-pointer merge seen in class. One pointer walks through each list:

- **intersection** (AND): if the two docIDs are equal the docID is added to the result and both pointers move on; otherwise only the pointer on the smaller docID moves on;
- **union** (OR): the smaller docID is always added to the result; at the end, the elements left in one of the two lists are appended.

Since both lists are sorted, the cost is $O(x + y)$, where x and y are the lengths of the lists. Queries with more terms are processed by merging the lists two at a time.

5.4 Optimized Conjunctive Queries

Before intersecting, the terms are sorted by **increasing document frequency**. In this way the intermediate result is small from the start and the following intersections are cheaper. The process stops as soon as the result is empty. A term that is not in the index has $df = 0$, so it is processed first and the empty result is found immediately.

5.5 Intersection with Skip Pointers

The algorithm is the same merge as before. The only difference is how a list moves forward: instead of moving by one position, it first tries to follow its skip pointers, as long as they do not go past the docID of the other list.

```
def _advance(lst, i, target):
    skipped = False
    while lst.skips[i] is not None and lst.postings[lst.skips[i]] <= target:
        i = lst.skips[i]
        skipped = True
    return i if skipped else i + 1
```

In the best case many elements are skipped and the cost becomes sublinear. In conjunctive queries with more terms, the intermediate result is also turned into a **SkipList**, so that every intersection can use skip pointers.

5.6 Spelling Suggestions (Edit Distance)

When a query term is not in the dictionary, the system suggests similar terms. The distance used is the **edit distance** (Levenshtein): the minimum number of insertions, deletions and substitutions of single characters needed to turn one string into the other. It is computed with the classic dynamic programming table in $O(M \cdot N)$, where M and N are the lengths of the two strings.

Comparing the term with all 48,404 terms would be slow, so only the terms **starting with the same letter** are compared (heuristic seen in class: errors rarely occur at the beginning of a word). The terms with the smallest distance are suggested, but only if the distance is at most 2. At most 5 suggestions are shown, sorted by document frequency. For example, for **brazl** the system suggests **brazil**.

6 Experiments

6.1 Skip Pointers

Setup. We measured only the time of the intersection between two postings lists: the lists are already loaded and the skip lists already built, since reading from disk is the same in both cases. We compared the plain intersection, skip pointers with span $\sqrt{P}$ and fixed spans from 2 to 64. Each measure is the average of 200 runs. The benchmark was run twice: the results were similar, except for one pair (*mln AND brazil*) which was noisy in the first run. Table 2 and Figure 2 show the second run.

Query	df	No skips	$\sqrt{P}$	Best fixed span	
said AND mln	15465 / 8625	1.950	3.264	2.681	(32)
oil AND price	1489 / 1920	0.348	0.456	0.448	(64)
cocoa AND coffee	129 / 230	0.031	0.054	0.044	(64)
said AND cocoa	15465 / 129	1.373	0.909	0.286	(8)
mln AND brazil	8625 / 435	0.847	1.245	0.521	(4)

Table 2: Intersection time in milliseconds.

Results.

- **Lists of similar length:** skip pointers make the intersection slower (from about 30% to 75%). The docIDs of the two lists are close to each other, so skips almost always fail, but the check is paid at every step anyway. In Python this extra work is even more expensive, because it is a function call.
- **One long list and one short list:** skip pointers help. With *said AND cocoa* the intersection is about 4.8 times faster with span 8. This is the case skip pointers are designed for: the long list can jump over large blocks of docIDs that are not needed.
- **The span matters:** the curve has a U shape. A small span gives many pointers but short jumps; a large span gives long jumps that often go too far and fail. The $\sqrt{P}$ heuristic is not always the best choice, because it only depends on the length of the list itself and not on the other list. For example, for *said* $\sqrt{P}$ gives a span of 124, which is too large.
- In our two favourable cases, the best span seems to grow with the ratio between the lengths of the two lists (ratio $\approx 120 \rightarrow$ best span 8; ratio $\approx 20 \rightarrow$ best span 4). With only two cases this is just an indication, not a rule.

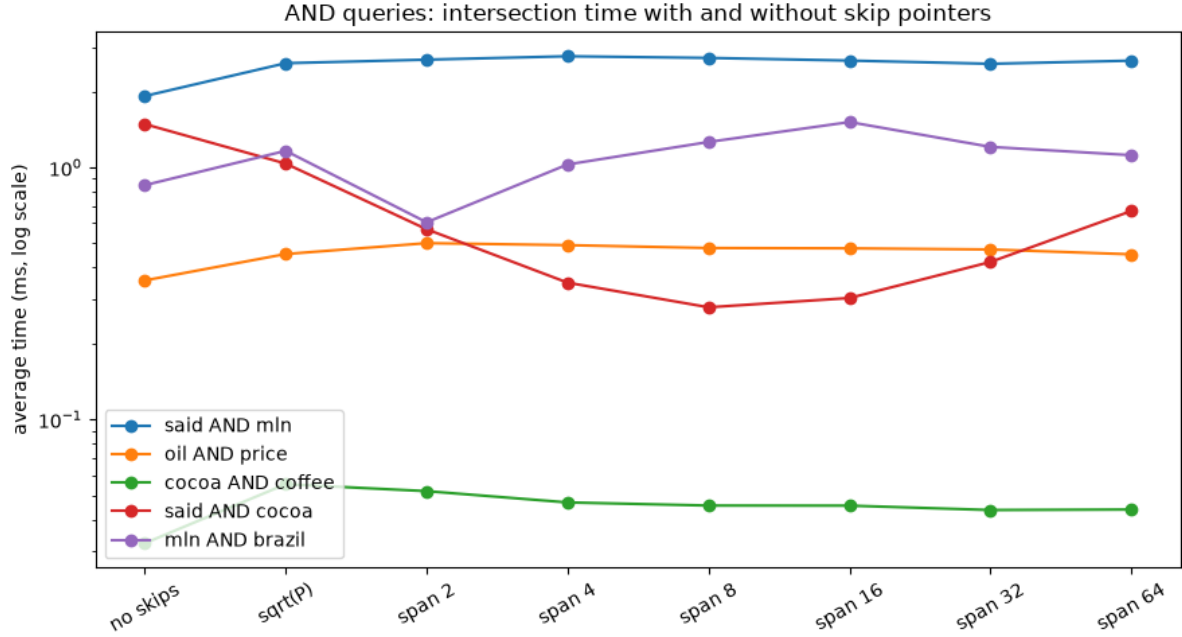


Figure 2: Intersection time with and without skip pointers (log scale).

6.2 Stop List

Table 3 shows the effect of the stop list on the size of the index.

	Without stop list	With stop list
Terms in the dictionary	48,429	48,404
dictionary.txt	849 KB	849 KB
postings.bin	6.7 MB	5.9 MB

Table 3: Size of the index with and without the stop list.

Removing only 25 terms (0.05% of the vocabulary) reduces the postings file by about 12%. This agrees with Zipf’s law: a few very frequent words have very long postings lists.

7 Limitations

- The tokenizer splits words like *U.S.* into *u* and *s*, and numbers like *155,221* into two tokens. In a query, *U.S.* is rejected as an invalid term.
- Queries cannot mix AND and OR, and NOT and parentheses are not supported.
- Spelling suggestions cannot fix an error in the first letter. Swapping two letters counts as 2 operations. Some “noisy” tokens of the dataset (e.g. *zzzz*, *braz*) can appear as suggestions.
- The whole index is built in memory, which is not possible for very large collections.
- Skip pointers are built when a query is run instead of being stored in the index.

8 Conclusion

We built a Boolean retrieval system based on an inverted index stored on disk, with a hashtable dictionary that points into a binary postings file. On top of it we added conjunctive and disjunctive queries, query optimization, a stop list, spelling suggestions and skip pointers. The experiments show that skip pointers are useful when the two lists have very different lengths, while they slow down the intersection of lists with similar length, and that the choice of the span has a large effect on the result.